In [52]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt
```

In [53]:
```
df=pd.read_csv("uber.csv")
```

In [54]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  str    
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  str    
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64  
dtypes: float64(5), int64(2), str(2)
memory usage: 13.7 MB
```

In [55]:
```
df.head()
```

Out[55]:

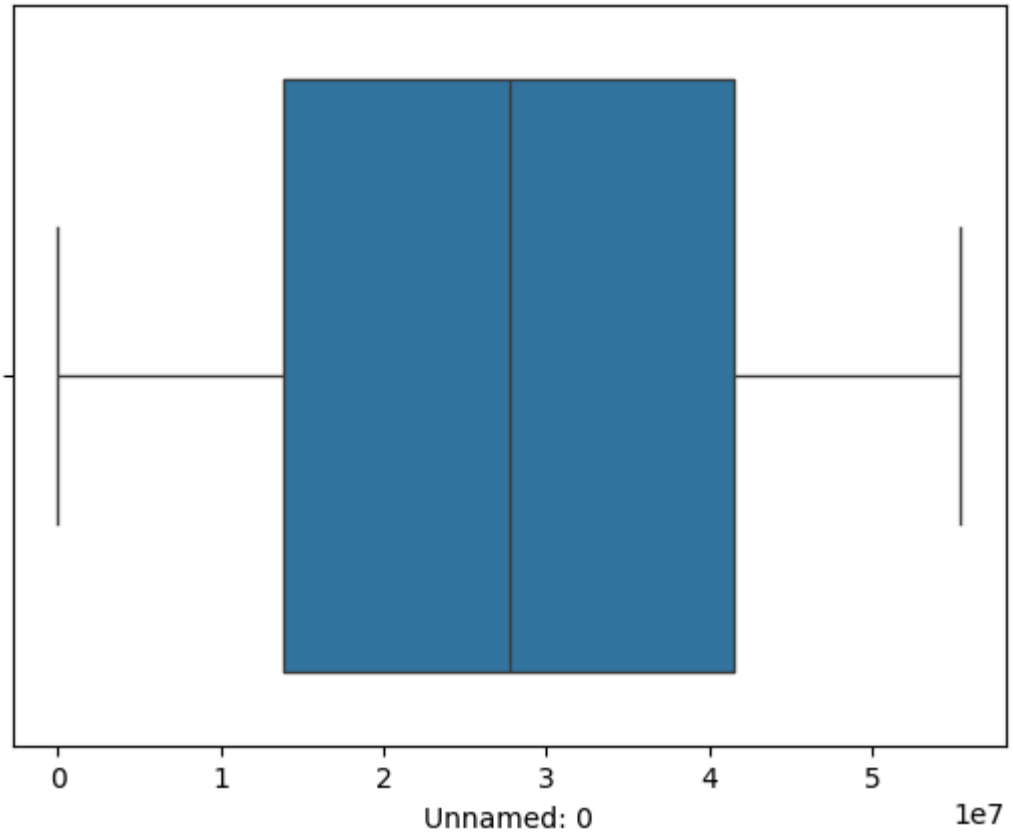
	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pick
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	

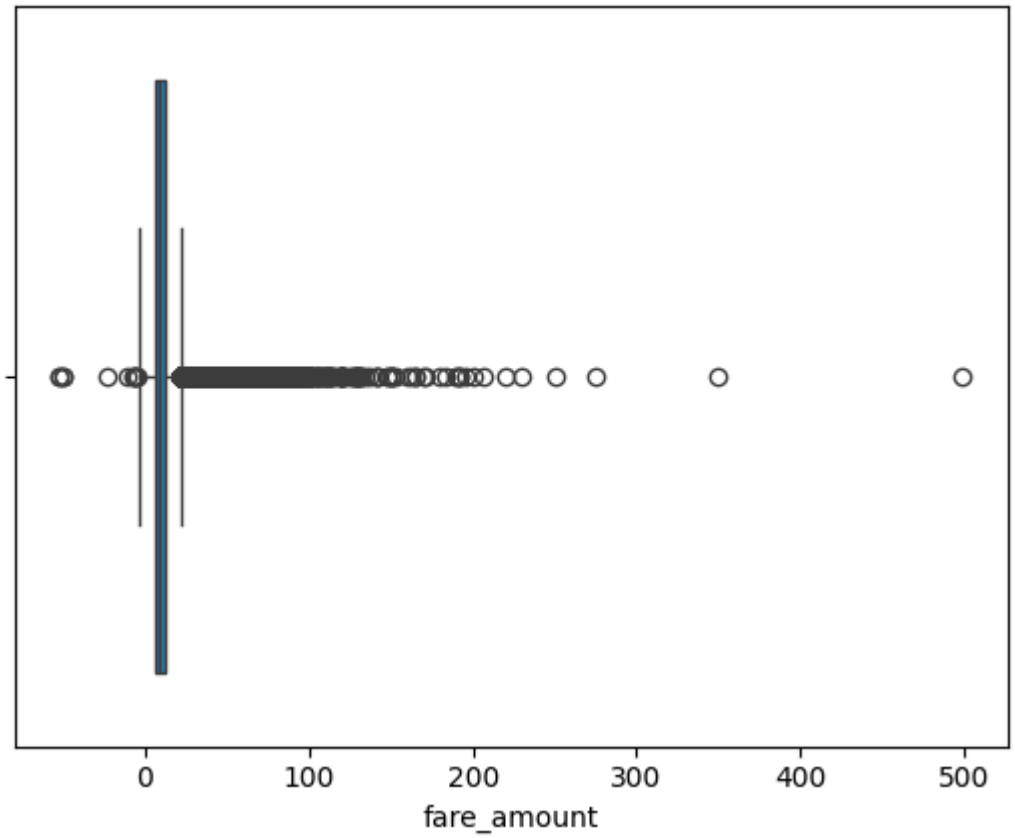
In [56]:
```
df.dropna(how="any",inplace=True)
```

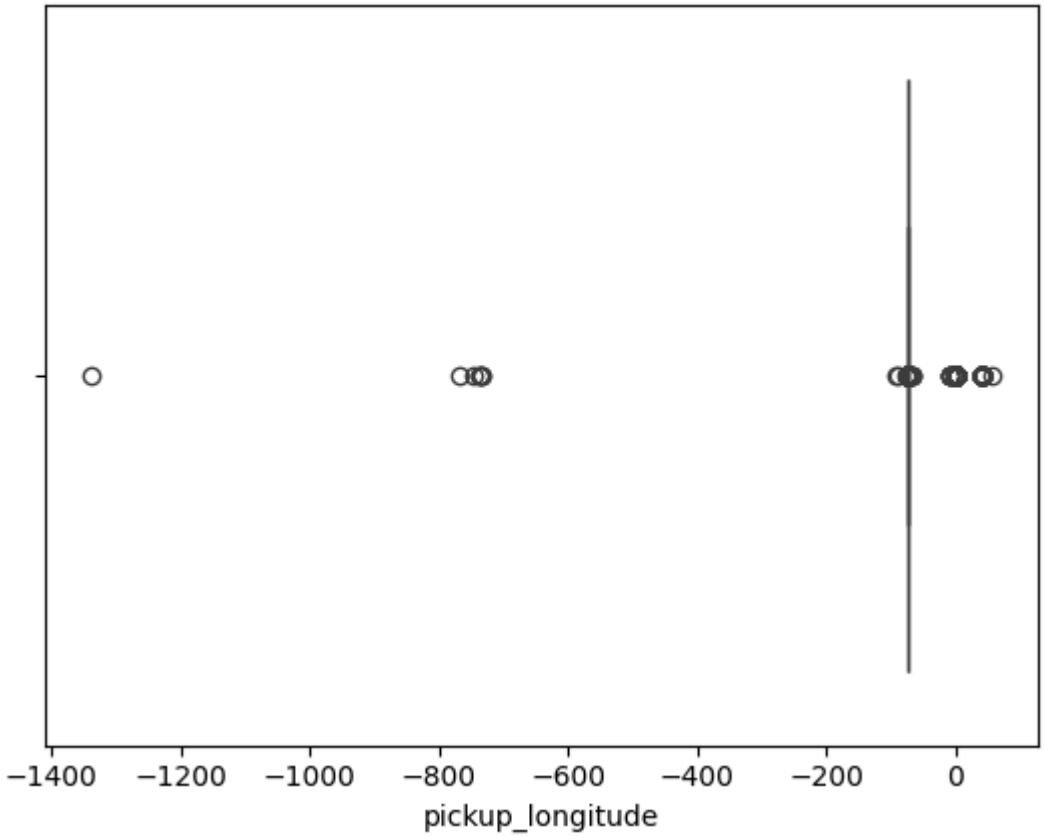
In [57]:
```
df.isnull().sum()
```

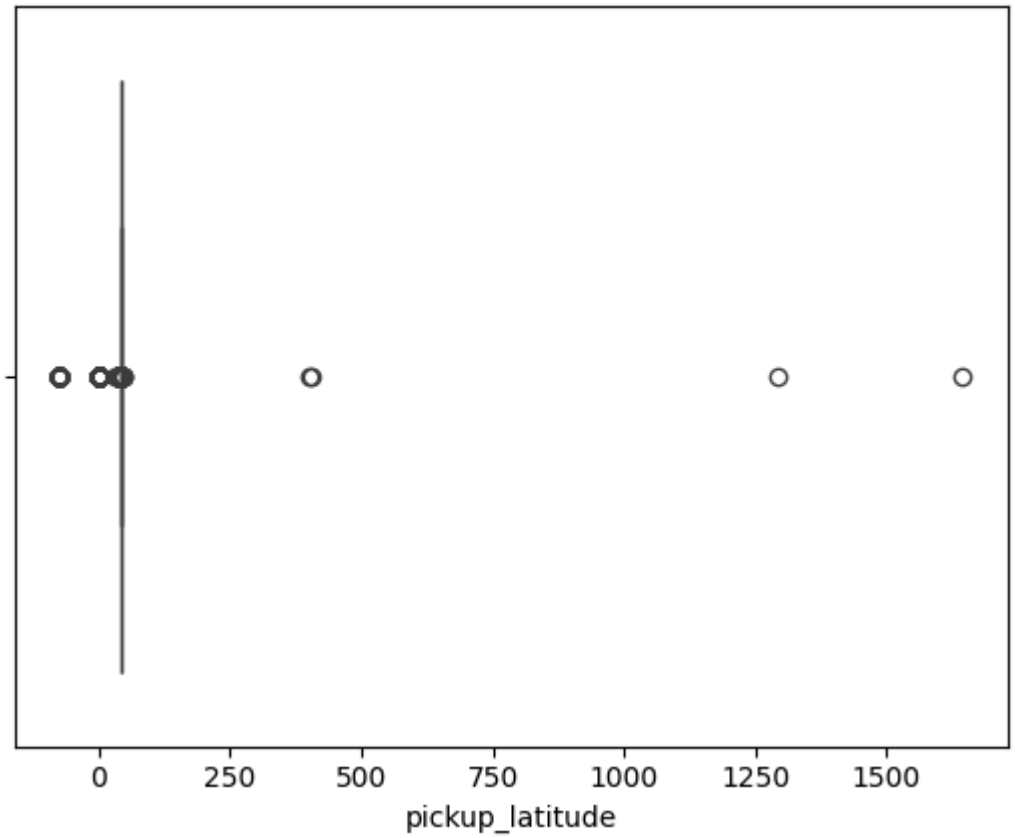
Out[57]:
```
Unnamed: 0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    0
dropoff_latitude     0
passenger_count      0
dtype: int64
```

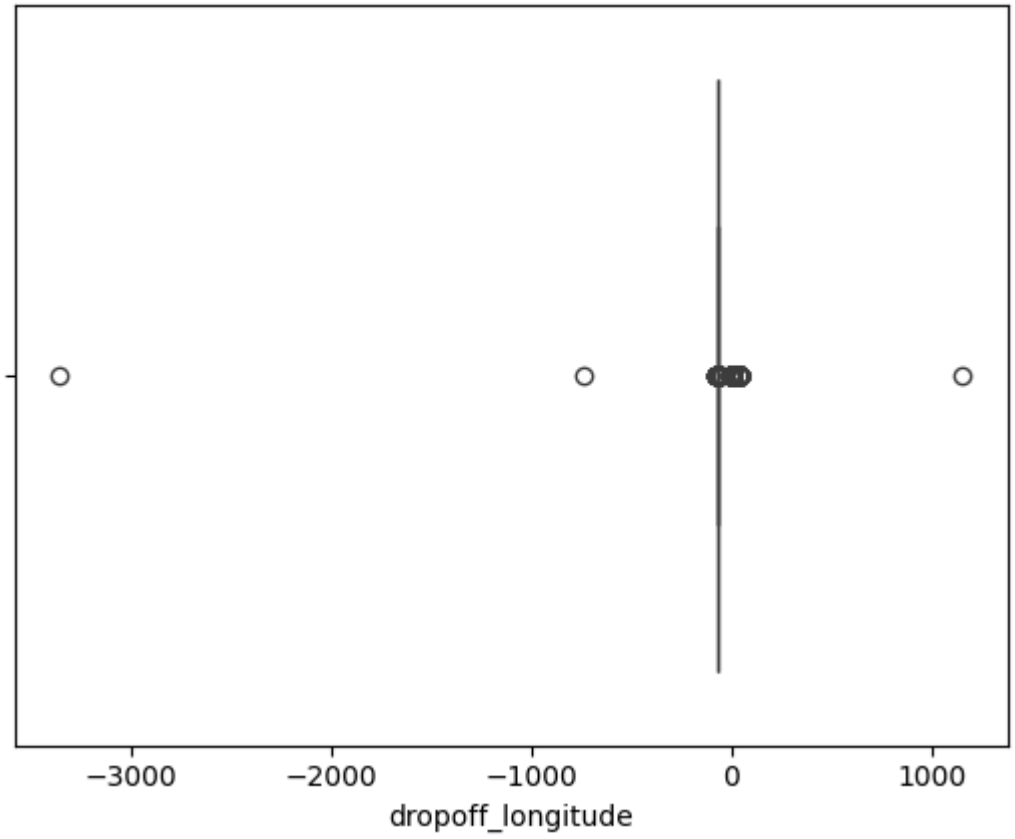
In [58]:
```
for col in df.select_dtypes(exclude=["object"]):
    plt.figure
    sns.boxplot(x=df[col])
    plt.show()
```

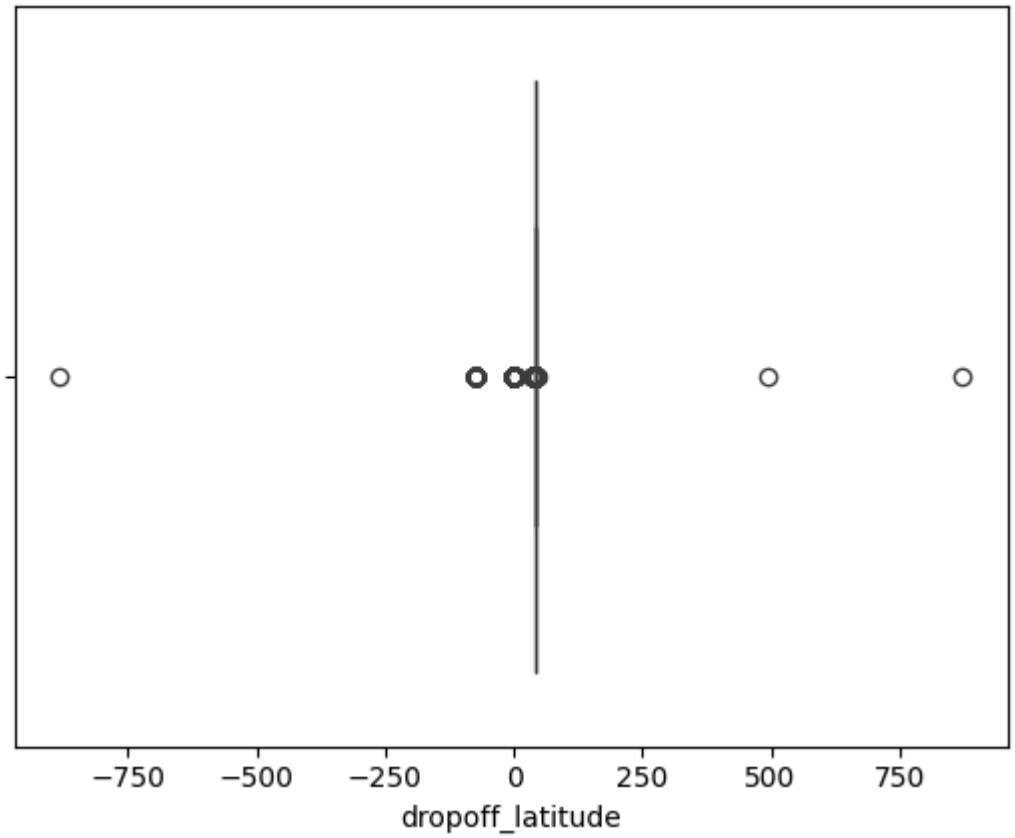


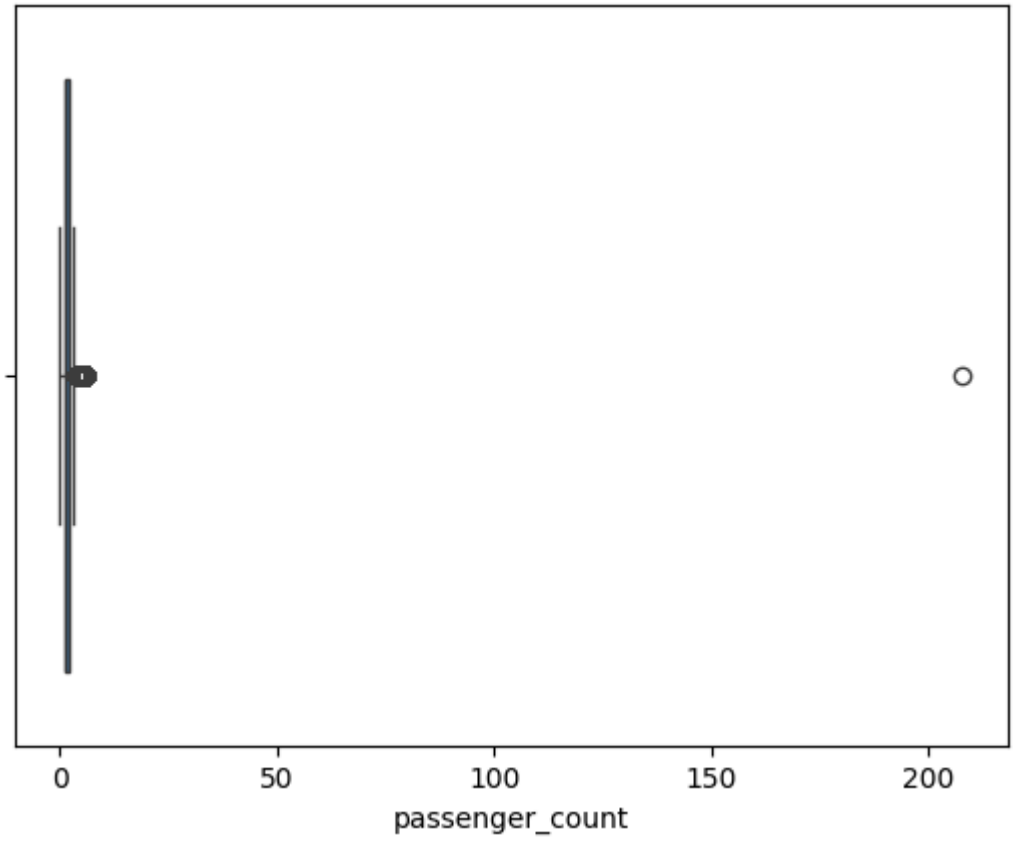

0
100
200
300
400
500
fare_amount


−1400
−1200
−1000
−800
−600
−400
−200
0
pickup_longitude


0
250
500
750
1000
1250
1500
pickup_latitude


−3000
−2000
−1000
0
1000
dropoff_longitude





In [59]:
```
df= df[
    (df.pickup_latitude >-90) &(df.pickup_latitude < 90)&
    (df.dropoff_latitude > -90) & (df.dropoff_latitude < 90)&
    (df.pickup_longitude > -180) & (df.pickup_longitude < 180)&
    (df.dropoff_longitude > -180) & (df.dropoff_longitude < 180)&
    (df.fare_amount > 0) & (df.passenger_count > 0) & (df.passenger_count < 50)
    ]
```

In [60]:
```
from math import cos, asin, sqrt, pi
import numpy as np

def distance (lat1, lon1, lat2, lon2):
    lon1,lon2,lat1,lat2 = map(np.radians, [lon1,lon2,lat1,lat2])

    diff_lon = lon2 - lon1
    diff_lat = lat2 - lat1

    km = 2*6371 * np.arcsin(np.sqrt(np.sin(diff_lat/2.0)**2 + np.cos(lat1)* np.c
    return km
```

In [61]:
```
df["distance_km"] = distance(df.pickup_latitude, df.pickup_longitude,
                             df.dropoff_latitude, df.dropoff_longitude)
```

In [62]:
```
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"], format="%Y-%m-%d %

df["is_weekend"] = df["pickup_datetime"].dt.dayofweek >= 5
df["day_type"] = df["is_weekend"].astype(int)

def time_of_day(hour):
    if 6 <= hour < 12:
        return "morning"
    elif 12 <= hour < 18:
        return "midday"
    else:
        return "night"

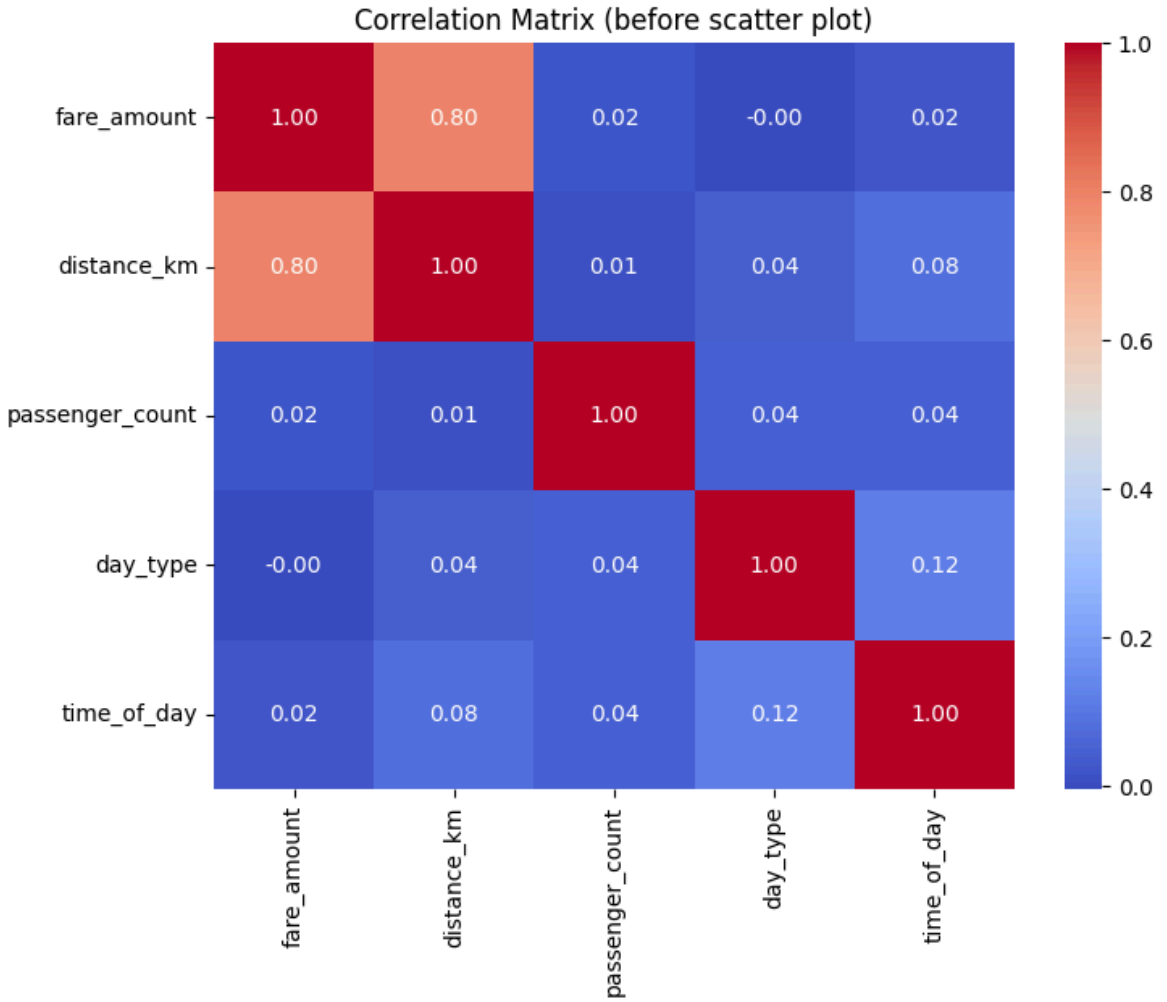
df["time_of_day_label"] = df["pickup_datetime"].dt.hour.apply(time_of_day)
df["time_of_day"] = df["time_of_day_label"].map({"morning": 0, "midday": 1, "nig
```

In [63]:
```
def remove_outliers_iqr(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[col] >= lower) & (df[col] <= upper)]

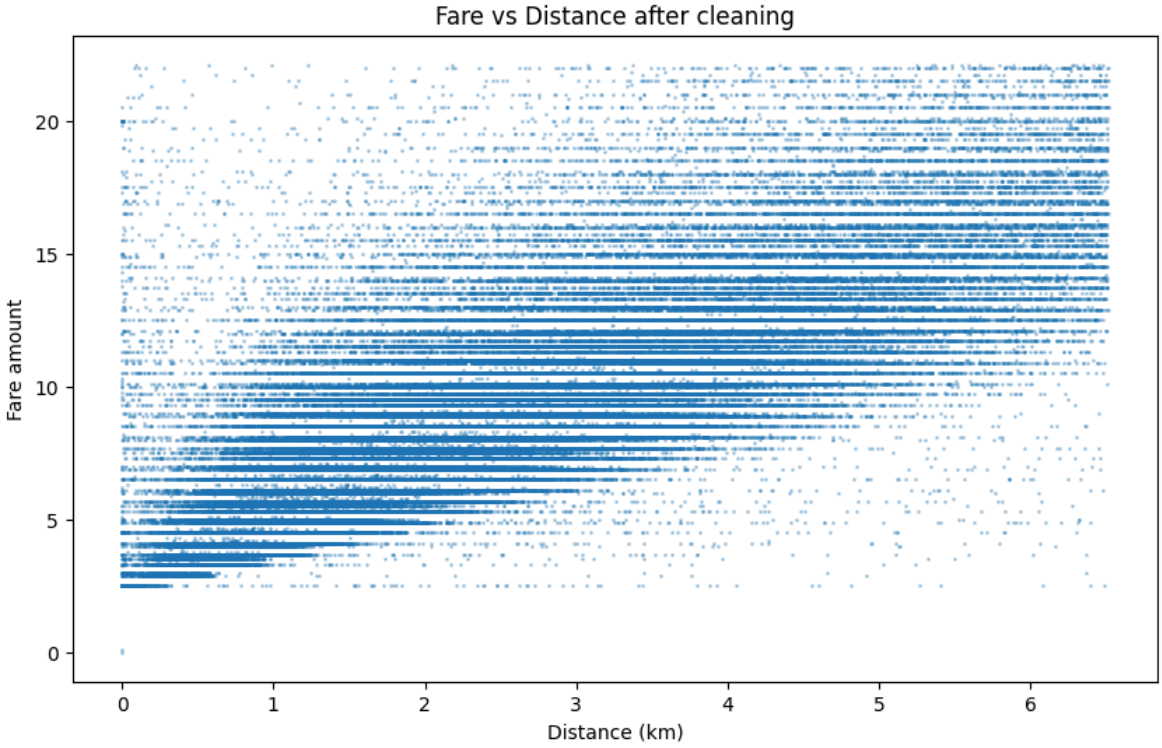
df = df[(df["fare_amount"] > 0) & (df["distance_km"] > 0)]

df = remove_outliers_iqr(df, "fare_amount")
df = remove_outliers_iqr(df, "distance_km")
```

In [64]:
```
corr_cols = ["fare_amount", "distance_km", "passenger_count", "day_type", "time_
plt.figure(figsize=(8, 6))
sns.heatmap(df[corr_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix (before scatter plot)")
plt.show()
```



In [65]:
```
plt.figure(figsize=(10, 6))
plt.scatter(df["distance_km"], df["fare_amount"], s=1, alpha=0.3)
plt.xlabel("Distance (km)")
plt.ylabel("Fare amount")
plt.title("Fare vs Distance after cleaning")
plt.show()
```



In [70]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

In [71]:
```
X = df[["distance_km"]]
y = df["fare_amount"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [72]:
```
lr = LinearRegression().fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

rf = RandomForestRegressor(n_estimators=100, random_state=42).fit(X_train, y_tra
y_pred_rf = rf.predict(X_test)
```

In [73]:
```
for name, y_pred in [("Linear Regression", y_pred_lr), ("Random Forest", y_pred_
    print(f"\n{name}")
    print(f"  R2:   {r2_score(y_test, y_pred):.4f}")
    print(f"  MAE:  {mean_absolute_error(y_test, y_pred):.4f}")
    print(f"  RMSE: {np.sqrt(mean_squared_error(y_test, y_pred)):.4f}")
```

```
Linear Regression
  R2:   0.6377
  MAE:  1.6126
  RMSE: 2.2387

Random Forest
  R2:   0.4661
  MAE:  1.9626
  RMSE: 2.7177
```

In [ ]: